

Aperture — System Architecture

Version: v1.0

Document Type: System Architecture

Related Documents

- Product Requirements Document
 - Development Roadmap
 - Technical Architecture
 - Low-Level Design
-

1. Purpose

This document describes the overall system architecture of Aperture.

Unlike the Product Requirements Document, which focuses on **what** Aperture should become, this document focuses on **how the platform is organized as a system**.

It intentionally avoids implementation details and specific code structures. Those belong in the Low-Level Design document.

Instead, this document defines:

- System boundaries
- Major architectural components
- Data flow
- Communication patterns
- Service responsibilities
- Domain decomposition
- Evolution strategy

The architecture is intentionally designed around an **interview-first** philosophy while remaining realistic for a production application.

2. Architectural Philosophy

Every architectural decision in Aperture follows four principles.

2.1 Simplicity First

The system should remain as simple as possible while satisfying current requirements.

Avoid introducing:

- distributed systems
- message queues
- microservices
- orchestration

until genuine product requirements justify them.

Complexity should emerge naturally rather than being designed prematurely.

2.2 Modular Design

Although Aperture initially ships as a modular monolith, every business capability should be isolated behind clearly defined domain boundaries.

Each domain should own:

- business logic
- persistence
- API endpoints
- background tasks
- validation
- documentation

The objective is to minimize coupling while avoiding distributed system overhead.

2.3 Product First

Infrastructure exists to support product features.

No technology should be introduced simply because it is commonly used in industry.

Every architectural addition must answer:

"What user problem does this solve?"

2.4 Evolutionary Architecture

The architecture should support gradual evolution.

Example:

Modular Monolith

↓

Background Workers

↓

Independent Services

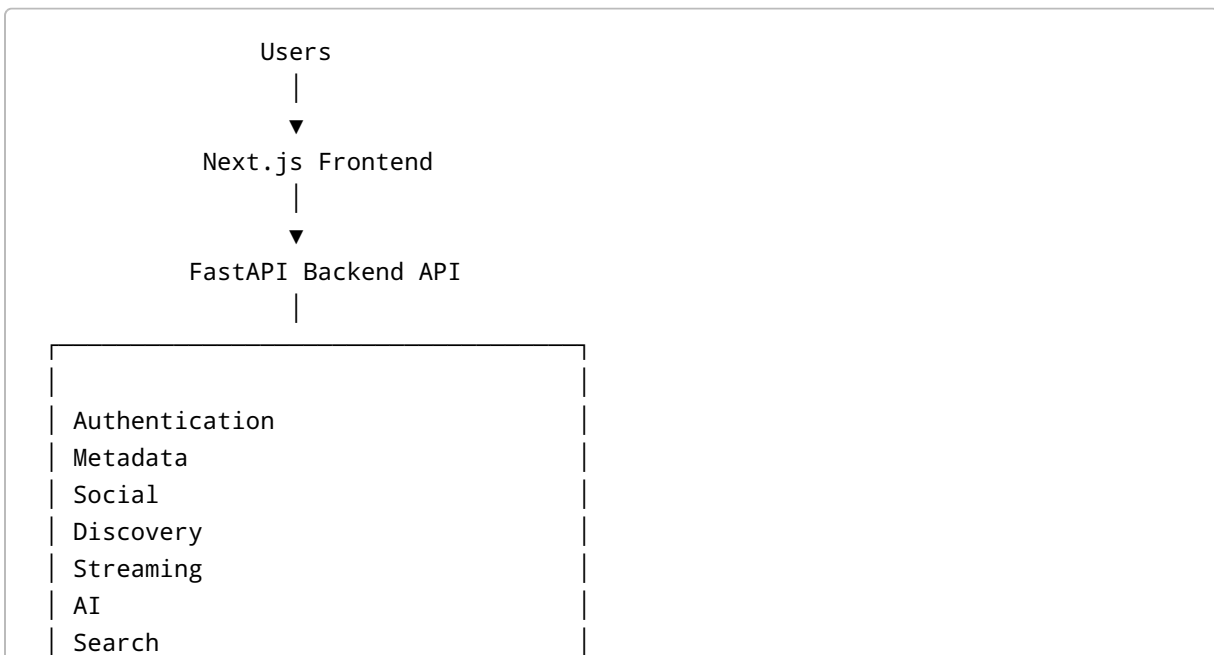
↓

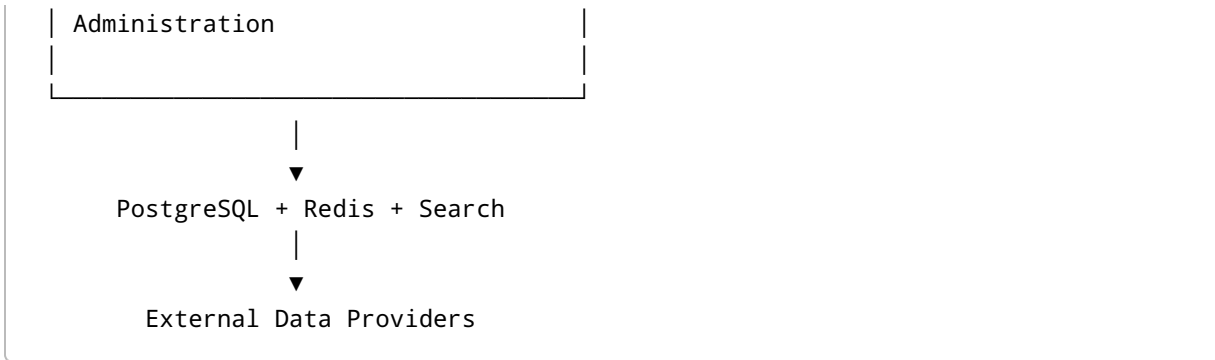
Distributed Platform

without requiring major rewrites.

3. System Overview

At its core, Aperture consists of five major subsystems.





This separation allows the backend to remain logically modular without introducing operational complexity.

4. System Context

The platform interacts with four external actors.

Users

Primary consumers.

Capabilities include:

- discovering content
- reviewing
- creating lists
- following users
- receiving recommendations

TMDb

Primary metadata provider.

Responsible for:

- movies
- TV series
- cast
- artwork
- watch providers
- trailers

The application never exposes TMDb directly.

Instead, all data is normalized into Aperture's internal model.

Authentication Providers

Initially:

- Google OAuth

Future:

- Apple
 - GitHub
-

Cloud Infrastructure

Responsible for:

- hosting
 - storage
 - CDN
 - deployment
 - monitoring
-

5. Architectural Layers

Aperture follows a layered architecture.

Presentation Layer

↓

Application Layer

↓

Domain Layer

↓

Infrastructure Layer

↓

Persistence Layer

Presentation Layer

Responsible for:

- UI
- accessibility
- routing
- animations
- client-side state

Contains no business logic.

Application Layer

Coordinates requests.

Responsibilities:

- validation
 - orchestration
 - authorization
 - transactions
-

Domain Layer

The heart of the application.

Contains:

- business rules
- services
- recommendation logic
- permissions
- workflows

No infrastructure code should exist here.

Infrastructure Layer

Responsible for:

- databases
 - Redis
 - search
 - external APIs
 - queues
 - cloud services
-

Persistence Layer

Responsible for:

- storage
 - indexing
 - caching
 - retrieval
-

6. Domain Decomposition

Instead of organizing by technology, Aperture is organized by business domains.

Authentication

Responsibilities

- login
 - registration
 - OAuth
 - sessions
 - permissions
-

Users

Responsibilities

- profiles
- preferences
- following

- cinematic identity
-

Metadata

Responsible for

- movies
 - TV
 - cast
 - crew
 - genres
 - synchronization
-

Discovery

Responsible for

- recommendations
 - trending
 - similarity
 - homepage
-

Reviews

Responsible for

- ratings
 - reviews
 - comments
 - likes
-

Lists

Responsible for

- watchlists
 - favorites
 - collections
 - rankings
-

Search

Responsible for

- indexing
 - autocomplete
 - semantic search
-

Streaming

Responsible for

- provider availability
 - regional availability
-

AI

Responsible for

- embeddings
 - cinematic DNA
 - taste matching
 - emotional discovery
-

Administration

Responsible for

- moderation
 - reporting
 - analytics
-

7. Request Lifecycle

Every request follows the same lifecycle.

Browser

↓

Frontend

↓

API Gateway

↓

Application Layer

↓

Domain Service

↓

Repository

↓

Database

↓

Response

This consistent request flow simplifies reasoning, debugging, and testing.

8. Metadata Flow

One of Aperture's defining architectural patterns is metadata ownership.

External providers never become the application's source of truth.

Instead:

TMDb

↓

Metadata Client

↓

Normalization

↓

Internal Model

↓

Database

↓

Frontend

Benefits:

- provider independence
- consistent schemas
- caching
- future extensibility

9. Search Flow

Search operates independently of transactional storage.

Metadata

↓

Indexer

↓

Search Engine

↓

Search API

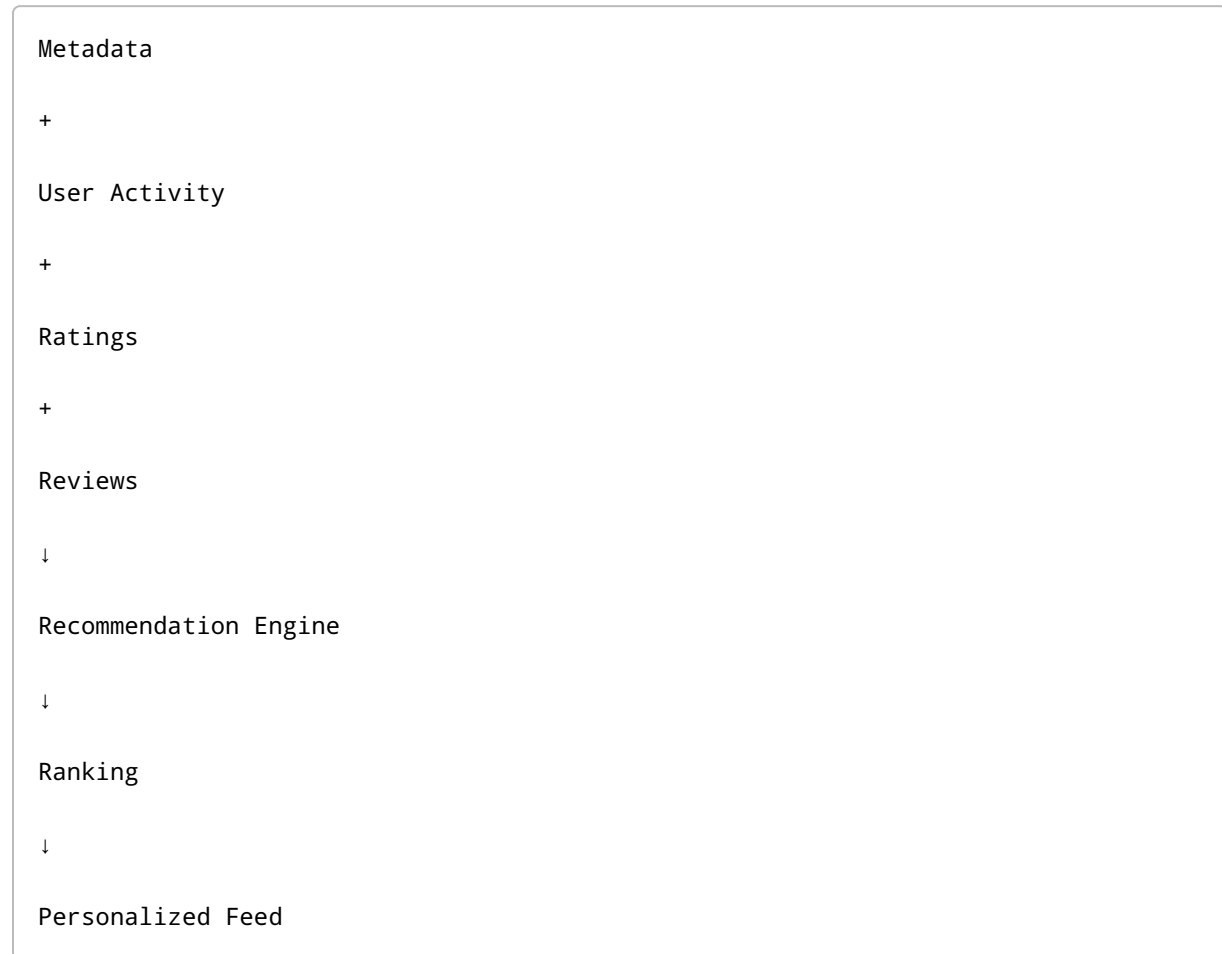
↓

Frontend

Separating search prevents expensive database queries from impacting user-facing workloads.

10. Recommendation Flow

Recommendations are generated from multiple independent signals.



No single signal determines recommendations.

This makes the system more resilient and adaptable.

11. Community Flow

Social interactions follow a publish-and-consume model.

User Action

↓

Activity

↓

Feed Generation

↓

Followers

↓

Notifications

This architecture enables future migration to asynchronous event processing without changing the user experience.

12. AI Integration Philosophy

Artificial intelligence is treated as an enhancement layer rather than a foundational dependency.

If AI systems fail, Aperture should remain fully functional.

Examples:

Without AI:

- metadata search works
- reviews work
- lists work

AI augments:

- recommendations
- similarity
- personalization

This reduces operational risk while allowing incremental improvements.

13. Scalability Strategy

Scalability is introduced progressively.

Stage 1

Single deployment.

Modular monolith.

Stage 2

Caching.

Background workers.

Search indexing.

Stage 3

Dedicated search.

Dedicated AI workers.

Dedicated ingestion workers.

Stage 4

Selective service extraction.

Only the highest-load domains become independent services.

This avoids premature microservices.

14. Security Boundaries

Authentication and authorization remain centralized.

Every request must pass through:

Authentication



Authorization



Validation



Business Logic

No business service should assume a request is already trusted.

15. Fault Tolerance

Failures should degrade gracefully.

Examples:

TMDb unavailable



Serve cached metadata.

Recommendation service unavailable



Fallback to trending content.

Search unavailable



Fallback to SQL search.

Graceful degradation is preferred over complete feature failure.

16. Architectural Evolution

The architecture intentionally evolves rather than being replaced.

Future additions include:

- distributed queues
- dedicated AI infrastructure
- event streaming
- read replicas
- CDN optimization

These additions should not require rewriting existing domains.

17. System Principles

Every new architectural decision should satisfy at least one of the following:

- Improves maintainability
- Reduces coupling
- Improves reliability
- Simplifies development
- Improves scalability
- Improves user experience

If a proposed change satisfies none of these, it should be reconsidered.

18. Summary

Aperture adopts a **modular, layered architecture** that prioritizes clarity, maintainability, and incremental evolution.

Rather than optimizing for hypothetical scale, the system is designed to grow alongside the product.

This architecture enables the project to demonstrate production-grade engineering practices while remaining approachable for a single developer and realistic for incremental implementation.